

What is an Array?

An **array** is a fixed-size, indexed collection of elements of the same type stored in contiguous memory.

Example:

```
int[] arr = {10, 20, 30};
```

Key Properties

- **Fixed Size:** Size is defined at initialization.
 - **Indexing:** Zero-based.
 - **Contiguous Memory:** Enables $O(1)$ access.
-

Array Declaration & Initialization

```
int[] a = new int[5];  
int[] b = {1, 2, 3};
```

9 Array Operations & Time Complexity

Operation	Time Complexity
Access by Index	$O(1)$
Search (Linear)	$O(n)$
Search (Binary, Sorted)	$O(\log n)$
Insert/Delete (Middle)	$O(n)$
Traversal	$O(n)$

Java Arrays Utility Methods (`java.util.Arrays`)

Common Methods

Method	Description	Time Complexity
<code>sort(array)</code>	Sorts full array	$O(n \log n)$
<code>binarySearch(array, key)</code>	Searches in sorted array	$O(\log n)$
<code>equals(arr1, arr2)</code>	Shallow equality check	$O(n)$
<code>deepEquals(arr1, arr2)</code>	Nested array equality	$O(n)$ recursively
<code>fill(array, val)</code>	Fill array with value	$O(n)$
<code>copyOf(array, newLength)</code>	Resize/copy array	$O(\min(n, newLength))$
<code>copyOfRange(array, from, to)</code>	Copy a range	$O(to - from)$
<code>asList(array)</code>	Convert to List	$O(1)$
<code>toString(array)</code>	String representation	$O(n)$
<code>deepToString(array)</code>	Nested to string	$O(n)$ recursively
<code>setAll(array, lambda)</code>	Functional value assignment	$O(n)$
<code>stream(array)</code>	Convert to stream	$O(1)$

Examples

```
int[] a = {1, 2, 3};
int[] b = Arrays.copyOf(a, 5); // [1, 2, 3, 0, 0]
Arrays.sort(a);
int index = Arrays.binarySearch(a, 2);
Arrays.fill(a, 0);
System.out.println(Arrays.toString(a));
```

Advanced Tips

- `System.arraycopy(src, srcPos, dest, destPos, length)` for fast copies.
- Use `Arrays.setAll()` for initialization with functions.
- `Arrays.asList()` returns a **fixed-size list** (modification not allowed).

Array Interview Patterns to Master

1. Two Pointers
 2. Sliding Window
 3. Prefix Sum
 4. Kadane's Algorithm
 5. Binary Search on Array
 6. In-place Modifications
 7. Frequency Maps / Counting Arrays
 8. Sorting + Searching
-

Practice Problems to Try

- Reverse Array
 - Rotate Array (in-place)
 - Find Missing Number
 - Majority Element (Moore's Voting)
 - Maximum Subarray (Kadane's)
 - Merge Sorted Arrays
 - Longest Subarray with Sum K
-

Summary Table

Feature	Array	ArrayList
Size	Fixed	Dynamic
Access	$O(1)$	$O(1)$
Insert/Delete	$O(n)$ (middle)	$O(n)$ (middle)
Memory	Compact	Uses resizing logic

Tip: Arrays are low-level, fast, and used in almost every DSA pattern. Master them deeply!

Prepared by ChatGPT for mastering Arrays in Java and cracking coding interviews.